

Business Report on Hotel Guest Review Classification using Natural Language Processing

Prepared by Nusrat

Scenario

A large hotel group runs many hotels in different cities. Every day the group receives a large number of guest reviews from websites and surveys such as Google reviews, TripAdvisor, Booking.com, the hotel website, and email forms. Each review is a short message in which a guest shares an opinion about the stay. Some reviews praise the staff, some complain about the room, and some report a booking mistake. The reviews arrive faster than the team can read them one by one, so the hotel wants a smart and quick way to understand each review and to act on it.

The Business Problem

The main problem is sorting. When a review arrives, the hotel needs to know which part of the service the review talks about, so it can send the message to the right team. At the moment a staff member has to read each review and decide where it belongs. This is slow, it costs money, and it can lead to mistakes when the team is busy. If reviews are sorted late or sorted wrongly, guests wait longer for a reply and small issues can grow into bad ratings.

In this project I solve this problem with text classification. I teach a computer to read a review and predict its service area. The six service areas are Staff Service, Booking Issue, Location, Check In, Breakfast, and Room Cleanliness. When the computer knows the area, the hotel can route the review to the correct department in seconds. This saves time, gives guests a faster reply, and shows the manager which areas receive the most complaints.

The Dataset

I have used a dataset of 120 hotel guest reviews. Each row is one review and the data has nine columns. The most important column is GuestReview, which holds the text the guest has written, and this is my input. The column ServiceArea holds the service area

of each review, and this is the label I want to predict. The other columns give extra details such as the review channel, the hotel city, the room type, the guest type, the stay date, and a star rating from one to five.

I have checked the data for quality. There are no missing values, so I do not need to remove or fill any rows. The target is also perfectly balanced, because each of the six service areas has exactly twenty reviews. A balanced dataset is good because the model does not favour one area over another.

Why I Choose ServiceArea as the Target

I choose ServiceArea as the target because it has six clear and balanced groups, which keeps the model fair and the results easy to judge. I have also looked at the Rating column from one to five, but its values are uneven and a short review does not always show the exact star score, so ServiceArea gives a cleaner and more useful task.

My Solution

To solve the problem, I build a Natural Language Processing pipeline in Python. A pipeline is a set of steps that take the raw review and turn it into a prediction. I explain each step below in simple words.

First I load the data and inspect it. I look at the size, the columns, the missing values, and the balance of the target, so I fully understand the data before I model it.

Second I clean the text. Raw text is messy, so I prepare it. I change every word to lowercase, I remove punctuation and numbers, I split the text into single words, I remove very common words such as the and was that carry little meaning, and I reduce each word to its base form. I save the result in a new clean text column. This step removes noise and keeps only the words that matter.

Third I explore the text. I count the most common words and I draw a chart of them. I also look at the top words inside each service area. This shows me that each area uses its own clear words. For example, the breakfast reviews use words like breakfast, coffee and buffet, while the check in reviews use words like wait and minute.

Fourth I study grammar and named items. I apply part of speech tagging to label each word as a noun, a verb or an adjective. I also apply Named Entity Recognition to find real world items such as city names, brands and amounts of time. For example the model finds Halifax and Niagara Falls as places and TripAdvisor as an organization.

Fifth I turn the words into numbers. A model cannot read words, so I use a method called TF IDF. It gives each word a score based on how important the word is inside a

review and how rare it is across all reviews. After this step every review becomes a row of numbers that a model can learn from.

Sixth I build and compare two models. I split the data into a training part and a testing part. I train the models on the training part and I test them on the testing part, which the models have not seen. I have been asked to compare two models, so I train Naive Bayes and Logistic Regression and I measure how well each one predicts.

Seventh I evaluate the models. I check the accuracy, I read the confusion matrix that shows where the predictions land, and I read the classification report that shows the score for each area.

The Output and Results

Both models give a very high accuracy on the test reviews, so they place almost every review in the correct service area. The confusion matrix shows that the predictions sit on the correct line, and the classification report shows a strong score for every area.

I want to be honest about this result. The accuracy is very high because the dataset is small, many reviews repeat, and each area uses very distinct words. This makes the task easy for the models. On a larger and messier set of real reviews the accuracy would likely fall, and the most common mistakes would happen between areas that share words, for example a staff comment that also mentions breakfast.

Because both models perform the same on this data, I keep Naive Bayes as my final model. I choose it because it is simple, fast, and easy to explain to a business team.

The table below gives a quick summary of the whole project.

Table:

Item	Detail
Dataset	120 hotel guest reviews with no missing values
Input text	GuestReview, the words written by the guest
Target	ServiceArea, with six balanced classes
Service areas	Staff Service, Booking Issue, Location, Check In, Breakfast, Room Cleanliness
Cleaning steps	lowercase, remove punctuation, tokenize, remove stopwords, lemmatize

Features	TF IDF, which turns words into numbers
Models compared	Naive Bayes and Logistic Regression
Final model	Naive Bayes
Result	Very high accuracy on the test reviews

Business Insight

One clear insight comes from the check in reviews. Many of them mention long waiting times measured in minutes. This tells the hotel that the check in step is a real pain point for guests. The hotel can act on this by adding staff at busy hours or by offering a faster online check in, which would reduce the wait and lift guest satisfaction.

Limitation

The main limitation is the data. The dataset is small and many reviews repeat, so the very high accuracy may not hold on new and varied reviews. The model has also learned from short and clean sentences, while real reviews can be long, mixed, and full of spelling mistakes. Before the hotel trusts the model in daily use, it should test the model on a larger and more realistic set of reviews and retrain it if needed.

Conclusion

In short, I have built a working pipeline that reads a hotel guest review and predicts its service area with strong results. The tool can save staff time and help the hotel reply to guests faster. With more and richer data it can grow into a reliable everyday helper for the service team.